

SCOPE: SCalable and Observable PERipheral-Borrowing Framework for FPGA-Based Prototyping

1st Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

2nd Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

3rd Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

Abstract—FPGA-based prototyping is critical for pre-silicon validation, but limited physical PCIe slots severely constrain multi-device topologies. Existing solutions either emulate peripherals in software, sacrificing physical fidelity, or rely on dedicated expansion/interconnect hardware, which makes fine-grained control transactions difficult to observe.

We present SCOPE, a scalable and observable peripheral-borrowing framework. SCOPE builds upon a reconfigurable FPGA routing substrate for cross-domain register and DMA operations that provides programmable DUT-to-host register-space remapping, reverse host-to-DUT translation for peripheral DMA, and DUT-visible interrupt regeneration. The substrate provides two control modes: Direct Control Mode maps device-register accesses in hardware for low-latency operation, whereas Proxy Forwarding Mode routes them through host software for command-level observation and intervention. A device-specific Semantic Adapter supplies the register, command-or-descriptor, DMA-address, completion, and event semantics required by each peripheral type.

We evaluate full NVMe data/control paths and demonstrate NIC register-access compatibility. Direct Control Mode increases host-side NVMe register-write latency by 14.0% over the FPGA-side NVMe baseline on ARM. The NVMe P2P data path retains up to 99.1% of host-memory sequential-read bandwidth. Proxy Forwarding Mode exposes an observable register-write latency of approximately 29 μ s. A virtual INTx test delivers every interrupt without timeout and measures a 956.81 ns mean host coordinator injection-path overhead.

I. INTRODUCTION

Field-Programmable Gate Arrays (FPGAs) serve as a critical platform for pre-silicon system-level validation and hardware-software co-development [?]. Unlike software simulators constrained to kHz-level frequencies and heavy abstraction [?], FPGA prototyping offers cycle-accurate, MHz-level execution. This enables early performance bottleneck identification and full software stack validation prior to tape-out, significantly accelerating SoC iteration cycles.

Modern CPU-centric SoC systems demand rigorous pre-silicon validation of the complete software ecosystem prior to tape-out. The Design Under Test (DUT) must boot production-grade operating systems and exercise hardware-level peripheral protocol semantics with comprehensive protocol compli-



Fig. 1. Comparison of conventional software peripheral simulation, FPGA prototyping with physical peripherals, and the proposed SCOPE peripheral-borrowing framework. SCOPE combines host-side real PCIe peripheral access with a direct P2P DMA data path and an observable proxy control path.

ance. This requirement for *protocol fidelity* distinguishes pre-silicon validation from software emulation and places stringent demands on both physical realism and observability.

Software-simulated endpoints can provide deterministic responses and high observability, but they cannot expose the DUT software stack to authentic hardware non-determinism, real PCIe link behavior, or device-specific driver corner cases [?]. Conversely, integrating high-performance PCIe peripherals into FPGA-based prototype environments reveals a fundamental physical bottleneck: limited on-board PCIe slots restrict realistic multi-device topologies such as RAID arrays or multi-NIC clusters.

TABLE I
DESIGN-SPACE POSITIONING OF PERIPHERAL INTEGRATION SCHEMES

Scheme	Real Endpoint	Transaction Path	DUT-visible Interface	Software-visible Commands	Dedicated Sharing Equipment
Software Emulation	No	Software-modeled	Emulated register/DMA interface	Yes	No
FMC/Local Attachment	Yes	Native PCIe path	Register/MMIO + DMA interface	No	FMC/slot
SmartIO Device Lending	Yes	NTB-forwarded PCIe path	Register/MMIO + DMA interface	No	NTB fabric
SCOPE Direct Control Mode	Yes	FPGA-routed PCIe path	Register/MMIO + DMA interface	No	No
SCOPE Proxy Forwarding Mode	Yes	P2P + proxy-replayed MMIO	Semantic command interface	Yes	No

Table ?? separates physical endpoint access from software-visible command semantics. Software emulation tools such as QEMU [?] and gem5 [?] expose modeled operations but do not exercise a real endpoint. Physical expansion via FPGA Mezzanine Cards (FMC) [?] (Figure ??) and Non-Transparent Bridging (NTB)-based SmartIO [?] (Figure ??) deliver hardware PCIe transactions but require dedicated expansion or lending hardware. SCOPE instead separates a device-to-DUT P2P data path from an optional software-visible control path while reusing the FPGA bridge and host-side peripherals already present in the prototyping environment.

figures/ntb_pcie_device_architecture_academic.drawio.png

Fig. 2. SmartIO architecture: Multi-host device sharing mechanism based on Non-Transparent Bridging (NTB).

To address this challenge, we propose SCOPE, an I/O-scalable and observable PCIe integration framework. SCOPE’s architectural contribution is the FPGA-side substrate that composes these facilities into a reconfigurable path across otherwise isolated DUT and host PCIe hierarchies. The main contributions are:

- **Reconfigurable FPGA routing substrate for cross-domain register/DMA operations:** SCOPE provides programmable DUT-AXI to host-PCIe register-space remapping for egress MMIO and reverse host-PCIe to DUT-AXI translation for ingress peripheral DMA. Together, these mappings decouple a peripheral’s physical

location from the address domains observed by the DUT and device.

- **Peripheral-semantic adaptation with selectable control modes:** SCOPE separates the reusable FPGA routing substrate from a device-specific adapter that describes submission registers, command or descriptor formats, DMA-address fields, completion conventions, and event-delivery policy. The adapter can configure Direct Control Mode for low-latency register access or Proxy Forwarding Mode to expose control fields before host-side submission, while interrupt regeneration converts host-captured events into DUT-visible IRQ signals.
- **Scoped system evaluation:** We implement the substrate on Xilinx UltraScale+ platforms. Same-device NVMe baselines quantify Direct Control Mode and P2P routing, proxy experiments quantify observable NVMe/NIC register writes, a virtual INTx benchmark validates DUT-visible event delivery, and synthetic concurrent streams characterize the shared ingress path. NIC evaluation is limited to register control-path compatibility.

II. BACKGROUND AND MOTIVATION

A. I/O Scalability Bottlenecks in FPGA Prototyping

Modern verification platforms increasingly adopt a Host-connected architecture [?] to leverage host computational resources for simulation offloading. However, despite improving management flexibility, this architecture faces severe I/O scalability bottlenecks. FPGA boards typically provide limited PCIe/M.2 interfaces, impeding the construction of multi-device topologies such as RAID arrays and multi-NIC clusters. While physical expansion via FMC [?] is a common remedy, it poses significant practical limitations. It requires dedicated expansion infrastructure (slots, cabling, power), introduces signal integrity risks where inter-board clock skew might obscure actual DUT bugs [?], and results in rigid topologies that often demand hardware rewiring or bitstream re-synthesis for test variations.

B. Related Work

To contextualize SCOPE within the broader landscape of FPGA-accelerated PCIe frameworks, we examine how existing solutions balance physical device access, data-path efficiency, and protocol visibility. DONGLE 2.0 [?] enables HLS accelerators to access either FPGA-attached or host-attached NVMe storage through a unified programming interface. Zero2M [?] moves NVMe queue management into FPGA hardware to

enforce multi-tenant storage QoS, while CeDMA [?] centralizes DMA control across heterogeneous accelerator nodes. These systems focus on application acceleration, scheduling, or storage-side computation. SCOPE instead targets processor-DUT validation by exposing host-side peripherals through a reconfigurable FPGA routing substrate while retaining hardware-level protocol semantics and an optional observable control path.

Device lending and virtualization provide complementary ways to share physical peripherals, but they make different trade-offs for validation. SmartIO [?] uses NTB adapters and pre-established address mappings to lend standard PCIe devices without software on the data path. SCOPE does not target lower latency than SmartIO; it targets FPGA DUT validation without a dedicated NTB lending fabric and adds an optional observable control path. VirtIO [?] and related virtualization interfaces abstract device operations into paravirtualized requests, whereas SCOPE exposes explicit hardware-level protocol semantics before requests reach the physical device. This visibility is important for driver development because verification engineers need to inspect details such as doorbell writes, queue pointer updates, and interrupt signaling sequences. Linux p2pdma [?] and QDMA P2P support enable peer-to-peer transfers within a single host or segment domain, but do not by themselves bridge the physically isolated DUT and host PCIe hierarchies.

SCOPE also differs from FPGA-based pre-silicon prototyping platforms. FireSim [?], FASED [?], and RAMP Gold [?] use FPGA acceleration to scale processor and on-chip-system simulation, with timing models or partitioned prototypes representing components under test. Their primary concern is simulation throughput and timing fidelity rather than attaching pools of real PCIe peripherals to a processor DUT. Resource-disaggregation systems such as DirectCXL [?] demonstrate that compute and memory placement can be decoupled through a routable interconnect. SCOPE applies this principle to peripheral validation: host-side PCIe devices are exposed through software-defined register/DMA routing, while the control/data-plane separation embodied by IX [?] keeps the high-throughput P2P path separate from the proxy path used for command-level debugging.

C. Design Insights and Architectural Opportunities

The challenges above motivate a shift from physical expansion to host-orchestrated peripheral routing. We derive two design insights:

- **Host-mediated control visibility:** By routing control flow through the host (Figure ??), SCOPE uses host software as a semantic intermediary. This grants command-level visibility for logging and analysis, while the low bandwidth of control traffic prevents this observability path from limiting the data plane.
- **Cross-domain zero-copy data movement:** By mapping DUT memory into the host PCIe address space (Figure ??), host-side peripherals can issue DMA transactions directly to DUT memory. The FPGA address translation

layer becomes a reconfigurable routing primitive that can support different peripheral types, including NVMe SSDs and NIC register control paths, without changing the datapath.

These insights form the foundation for the system architecture proposed in the subsequent section.

III. ARCHITECTURE OF SCOPE FRAMEWORK

SCOPE is an FPGA routing substrate rather than a new PCIe controller, DMA engine, or user-space I/O library. Its FPGA-side mechanisms are: (1) programmable egress remapping from DUT AXI addresses into host PCIe Base Address Register (BAR) space; (2) reverse ingress translation from host PCIe bus addresses to DUT AXI memory; (3) a selectable insertion point that routes BAR control either directly through hardware or through an observable host proxy; and (4) interrupt regeneration that presents host-captured device events as DUT-visible IRQ signals. These mechanisms coordinate the existing PCIe IP and software stack around the address domains and observability requirements of FPGA-based processor validation.

Consistent with the control-data plane separation philosophy espoused by IX [?], SCOPE explicitly decouples operation into two planes. The *control plane* is responsible for semantic routing decisions such as address translation, command interception, and protocol analysis, while the *data plane* handles high-throughput data movement via P2P DMA. This separation ensures that observability enhancements on the control plane do not compromise data-path performance.

By analyzing the traditional PCIe device model, the interaction is decomposed into three core components:

- **Memory Mapped IO (MMIO):** For accessing control registers to configure the device.
- **Direct Memory Access (DMA):** For high-efficiency data transfer between the Host and the device.
- **Interrupt:** For event notification from the device to the Host.

To enable the DUT to access host-side peripherals, the substrate realizes three mechanisms: *Control Transaction Propagation*, *Cross-Domain DMA*, and *Interrupt Substitution*. Their contribution is the coordinated cross-domain routing behavior, not the underlying standard PCIe, DMA, or user-space I/O components in isolation.

A. Control Transaction Propagation Mechanism

The core of the control path ensures reliable delivery of DUT-initiated MMIO operations. To accommodate diverse verification requirements, we design two control modes.

Direct Control Mode requires SCOPE-aware DUT-side drivers, while Proxy Forwarding Mode uses an explicit split-driver control plane

1) *Direct Control Mode:* In performance-sensitive scenarios requiring minimal latency (**performance validation**), Direct Control Mode uses a direct mapping strategy. As shown in Figure ??(a), the system maps host peripheral registers into the DUT's physical address space. DUT-side software then issues

figures/Implement_Overview_ieee.png

Fig. 3. Overall Architecture of the SCOPE Framework. (a) In Direct Control Mode, the control and data paths use bridge modules for low-latency cross-domain protocol conversion. (b) In Proxy Forwarding Mode, the control and data paths use bridge modules for packet-based command transmission.

MMIO operations carrying the device's hardware-level BAR semantics through this mapped aperture. Direct Control Mode focuses on low-latency BAR access through explicit DUT-side software configuration rather than OS PCIe enumeration of the remote peripheral.

The key abstraction is that command submission for many PCIe peripherals is initiated by writes to device BAR space. The device-specific adapter identifies the relevant submis-

sion register and its semantics, while the common mapping table routes the resulting MMIO transaction. For example, NVMe uses submission/completion queue doorbells, whereas a descriptor-ring device can use a queue producer or tail register. Retargeting this control operation changes the adapter configuration rather than the FPGA bridge datapath.

However, a challenge arises at the OS level: since the remote device is not attached to the DUT's local Root

Complex, standard bus enumeration fails, rendering kernel-based management ineffective. To resolve this, we propose a **Kernel Bypass Access Paradigm**. By mapping the control path directly to the user virtual address space, a lightweight user-space driver manages device configuration, effectively circumventing the kernel’s logical identity validation while ensuring reliable control.

2) *Proxy Forwarding Mode*: For scenarios requiring granular observability (System Behavior Analysis), we propose a **Proxy Forwarding Mode**. As illustrated in Figures ??(b) and ??, the host coordinator handles control commands initiated by the DUT.

Instead of direct hardware interaction, the DUT-side proxy driver serializes register operations into command packets for the host. The host coordinator parses these requests, executes the actual MMIO operations, and returns the response. The common proxy request carries the destination BAR offset and write value; the semantic adapter may additionally decode or snapshot the associated command or descriptor when deeper observability is required. Descriptors need not be copied into the proxy path when the physical device can fetch them from DUT memory through cross-domain P2P DMA. Proxy Forwarding Mode intentionally trades control-path latency for command visibility and intervention; it is intended for device bring-up, command tracing, and fault analysis. Direct Control Mode is used when control-path latency or peripheral performance is the evaluation target.

B. The Cross-Domain DMA Mechanism

To achieve high-efficiency, zero-copy data transmission, we propose a routing mechanism based on **PCIe P2P technology**. The core concept involves exposing the DUT’s physical memory to the host address domain via BARs and configuring the peripheral’s DMA target to point to this space.

To ensure integrity, an **Address Domain Translation** mechanism within the FPGA remaps ingress TLPs from the Host domain to the DUT’s local AXI domain. Simultaneously, Protocol Translation logic converts PCIe Memory Read/Write TLPs into DUT-local AXI transactions. This seamless redirection ensures that peripheral-initiated DMA requests are correctly routed to the DUT memory, bypassing Host memory buffering entirely.

C. Interrupt Substitution Mechanism

Since host-side interrupts cannot physically reach the DUT, an Interrupt Substitution Mechanism is required to maintain system responsiveness. We design two schemes catering to different driver architectures.

1) *Polling-Based Substitution*: For high-throughput user-space applications, we utilize a polling approach. As illustrated in Figure ??, the DUT driver actively monitors the Completion Queue (CQ) in memory for status updates written by the device. This eliminates kernel context switching overhead, optimizing performance for throughput-sensitive workloads [?].

figures/.drawio.png

Fig. 4. Polling-Based Substitution Mechanism for latency-sensitive status monitoring

2) *Interrupt Source Modification*: For interrupt-driven software, we implement an “Interrupt Source Modification” scheme. As depicted in Figure ??, the host coordinator captures physical MSI-X interrupts and encapsulates them into messages sent to the FPGA. Internal logic then decodes these messages and asserts physical IRQ lines to the DUT processor, providing a DUT-visible event-delivery path compatible with its configured interrupt handler.

D. Peripheral Adaptation Methodology

SCOPE separates a reusable cross-domain FPGA routing substrate from a device-specific semantic adapter. The FPGA routing substrate retains the same BAR routing, reverse DMA translation, control-mode selection, proxy transport, and interrupt-regeneration mechanisms across peripheral classes. Adapting a PCIe peripheral requires describing five elements:

- 1) **Submission trigger**: the BAR register and write semantics that notify the device of new work, such as a doorbell or queue-tail update.
- 2) **Work description**: the command, queue entry, or descriptor format associated with the submission trigger.
- 3) **DMA addressing**: the fields that identify DUT-resident payload and metadata buffers and must be mapped into the host PCIe address domain.
- 4) **Completion convention**: the completion queue, descriptor write-back, or status field through which the device reports progress.
- 5) **Event policy**: whether completion is observed by polling or delivered through the interrupt-regeneration path.

These elements define the boundary between common FPGA routing and device-specific protocol semantics. In Di-

figures/Interrupt_sub.drawio.png

Fig. 5. Architecture of the Interrupt Source Modification Mechanism.

rect Control Mode, the adapter configures the BAR aperture and DMA-address mappings used by the DUT-side software. In Proxy Forwarding Mode, it identifies the control operation to intercept and, when required for debugging, the command or descriptor fields to decode. NVMe realizes this model through SQ/CQ doorbells, command entries, PRP/SGL addresses, and completion queues. A modern NIC follows the same DMA-driven structure using descriptor rings, queue producer or tail registers, buffer addresses, and completion write-back.

IV. IMPLEMENTATION

A. Overview

SCOPE maps the two control modes onto two decoupled PCIe paths on Xilinx UltraScale+ devices. Direct Control Mode uses the QDMA bridge path for AXI-PCIe conversion and programmed BAR/address remapping, while Proxy Forwarding Mode uses an XDMA stream path to exchange command packets with the host coordinator. The FPGA routing substrate is device-independent; device-specific software supplies the semantic adapter for BAR triggers, descriptor fields, DMA addresses, completions, and events.

B. Control Path Implementation

In Direct Control Mode, DUT-side software maps the FPGA aperture through `mmap` and issues load/store accesses to retargeted BAR registers. The address-mapping table translates DUT AXI addresses into host PCIe addresses, allowing the same bridge path to reach an NVMe BAR or NIC BAR by changing the target base and offset.

In Proxy Forwarding Mode, the DUT-side proxy driver serializes intercepted BAR operations into DMA packets. The

figures/proxy_forwarding_software_architecture_rev

Fig. 6. Split-Driver Software Architecture for Proxy Forwarding Mode. The system decouples the control logic from physical transmission through a three-layer abstraction: Protocol, Transport, and Physical Interface.

host coordinator receives these packets through the XDMA stream interface and dispatches them to a protocol layer built on DPDK/SPDK [?], [?], [?]. The implemented NVMe adapter decodes command capsules before submission, while a generic BAR-write adapter forwards offset/value tuples for NIC BAR-control evaluation.

C. Data Path Implementation

The data path separates device-specific queue preparation from generic FPGA routing. The host coordinator rewrites DUT-local buffer addresses into host-side PCIe bus addresses before submitting device commands. For NVMe, this requires parsing and rewriting PRP entries. When the device issues DMA, the QDMA bridge receives the resulting TLPs and the FPGA ATU translates them back into the DUT AXI address domain. Descriptor-ring devices can reuse the same FPGA routing substrate once their semantic adapter identifies the relevant buffer-address fields.

D. Interrupt Substitution Implementation

Polling-based operation uses SPDK-style completion-queue polling: the DUT software checks CQ phase tags, processes completed entries, and advances the head doorbell without registering an ISR. For interrupt-driven operation, a host kernel module captures MSI-X events [?] and sends compact interrupt packets through the control DMA path; FPGA logic decodes the vector and asserts the corresponding DUT-visible IRQ line.

Virtual INTx Test Endpoint: For controlled characterization, we implement a virtual INTx endpoint with `TRIGGER`

and CLEAR registers in BAR0. TRIGGER requests host coordinator injection through the FPGA interrupt path, and CLEAR deasserts the level after the DUT handler runs. The endpoint isolates host-to-DUT injection and regeneration rather than physical-device MSI-X capture latency.

V. EXPERIMENTAL EVALUATION

The evaluation isolates the measured behavior of the implemented FPGA routing substrate. Micro-benchmarks quantify BAR control, virtual INTx injection, and NVMe P2P data paths, end-to-end tests exercise NVMe operation on hard-core and soft-core DUTs, and synthetic concurrent Memory Write ingress streams stress the shared FPGA Memory-Write ingress path. NIC measurements cover BAR control-path compatibility rather than packet TX/RX or NIC DMA.

A. Experimental Setup

The experimental platform is anchored by an Intel Xeon Gold 6248R host, enterprise-grade NVMe SSDs (Intel P4510), and a host-side PCIe NIC. We employ two FPGA platforms and three DUT implementations for complementary evaluation objectives:

- **ARM Cortex-A53 on XCZU19EG:** The 1.2 GHz hard core evaluates Direct Control Mode and the high-throughput P2P path without making a low-frequency soft core the primary limiter.
- **RocketChip on XCZU19EG:** The 100 MHz configurable RISC-V soft core [?] evaluates end-to-end operation on a processor instantiated in the FPGA fabric.
- **XiangShan Nanhu-G on VU37P:** The 50 MHz out-of-order RISC-V DUT [?] evaluates the observable Proxy Forwarding Mode. The VU37P provides sufficient logic and memory resources for Nanhu-G and the proxy transport path.

This division demonstrates that SCOPE's transaction-routing abstraction applies across multiple DUT implementations.

To instantiate the dual-mode architecture, the hardware incorporates two decoupled PCIe IP cores. The **Xilinx QDMA [?] Subsystem**, configured in Bridge Mode, serves as the AXI-PCIe conversion bridge for Direct Control Mode and the P2P data path. The **Xilinx XDMA [?] Subsystem**, deployed in Stream Mode, functions as a packet DMA engine for Proxy Forwarding Mode, isolating proxy control traffic from the high-throughput data path. On the software side, the stack features a custom SPDK-based user-space driver and a specialized fabric transport layer to implement address mapping and interrupt handling. The interrupt benchmark uses a DUT-side proxy driver and the virtual INTx endpoint described in Section ??.

B. Micro-benchmark Analysis

1) *Control Path Latency:* We evaluate control-plane latency across two control modes: Direct Control Mode register access and Proxy Forwarding Mode register-write access.

1. **Direct Control Mode Register Access:** We measure DUT-observed MMIO-write latency for peripheral BAR registers directly mapped into the DUT address space. The interval begins when the DUT issues the write request and ends when it receives the corresponding AXI B response; it therefore measures the requester-visible mapped-write path rather than the device's internal completion time. Table ?? shows that the ARM hard core takes 1295.11 ns to write the host-side NVMe BAR, 159.11 ns (14.0%) above the 1136.00 ns FPGA-side NVMe baseline. RocketChip takes 1616.35 ns for the host-side NVMe BAR, 477.78 ns (42.0%) above its 1138.57 ns FPGA-side baseline; this absolute value also includes the request path of the 100 MHz soft core. Retargeting the same mapping to the host-side NIC yields 1143.38 ns and 1432.90 ns for ARM and RocketChip, respectively. The FPGA board has no locally attached NIC, so an FPGA-side NIC baseline is unavailable.

TABLE II. Direct Control Mode Register MMIO Write Latency

Processor	FPGA-side NVMe	Host-side NVMe	Host-side NIC
ARM A53	1136.00 ns	1295.11 ns	1143.38 ns
RocketChip	1138.57 ns	1616.35 ns	1432.90 ns

2. **Proxy Forwarding Mode Register-Write Access:** We evaluate Proxy Forwarding Mode on a 50 MHz XiangShan Nanhu-G DUT using the same procedure for the NVMe and NIC BAR targets. The FPGA starts a 250 MHz latency counter when it receives the Nanhu-G BAR-write request, encapsulates the request for the host proxy, and stops the measurement when Nanhu-G receives the returned AXI B response after the proxy has accessed the real device BAR. Thus, the reported value is a DUT-observed time from request arrival at the FPGA to write-response delivery, with a 4 ns counter resolution. Each target is measured over 1000 operations.

Table ?? reports mean proxy-path latencies of 29.92 μ s for the NVMe BAR and 28.06 μ s for the NIC BAR. As a reference point, NVMe-oF TCP is a software-based remote NVMe transport that carries NVMe commands over standard TCP/IP networks, and is commonly used as a practical low-latency baseline for remote storage access. SCOPE's proxy-path latency is slightly lower than the reported NVMe-oF TCP latency of 31.52 μ s [?]. This result indicates that Proxy Forwarding Mode can expose BAR-level control events for command tracing while keeping the added delay within the range of an accepted remote-I/O software path. Therefore, Proxy Forwarding Mode is sufficient for system behavior analysis.

TABLE III. Proxy Forwarding Mode BAR-Write Latency on XiangShan Nanhu-G

Target BAR	Mean Latency
NVMe BAR	29.92 μ s
NIC BAR	28.06 μ s

2) *Virtual INTx Injection Path:* We validate DUT-visible event delivery with the virtual INTx endpoint. The DUT-side proxy driver performs 100 trigger-and-clear interrupt cycles,

while the host coordinator records the adjusted injection fast-path latency. All 100 interrupts reached the DUT-side handler without timeout or unexpected events. The mean adjusted host coordinator injection latency is 956.81 ns, which is only 3.2% of the 29.92 μ s NVMe proxy BAR-write latency. This result shows that virtual INTx regeneration adds negligible latency relative to proxy BAR forwarding.

3) *Data Path Performance (P2P DMA)*: This experiment evaluates the efficiency of the PCIe P2P data path by comparing DMA transfers from host-side NVMe devices to FPGA memory versus host memory.

Bandwidth & IOPS: Figure ?? compares the P2P path with a host-memory target using the same NVMe device. The FPGA-memory path achieves 2596.3 MB/s versus 2620.5 MB/s for 128K sequential reads, retaining 99.1% of the baseline, and 236777.4 versus 240372.3 IOPS for 4K random reads, retaining 98.5%. For writes, it achieves 729.5 versus 1023.3 MB/s and 188433.4 versus 244780.9 IOPS, retaining 71.3% and 77.0%, respectively. The read/write asymmetry is consistent with back-pressure caused by finite posted-data credits and write-acceptance buffering in the current bridge implementation. We have not yet instrumented credit occupancy at each stage, so this result identifies a likely bottleneck rather than proving that credit exhaustion is the sole cause.

Write-Path Optimization: The current write gap is an implementation limitation rather than a requirement of the cross-domain routing model. Planned optimizations include increasing PCIe receive buffering and the advertised posted-data credit budget where supported by the IP configuration, decoupling PCIe ingress from AXI back-pressure with deeper queues, increasing the number of outstanding AXI writes, and tuning payload size and AXI burst formation to reduce per-transaction overhead. We will evaluate these changes with PCIe flow-control counters and matched TLP/AXI traces, then repeat the same-device bandwidth and IOPS tests to isolate the contribution of each optimization.

C. Synthetic Ingress Scalability Analysis

Having established the baseline efficiency of the P2P data path, we further characterize the scalability of the shared FPGA ingress path under concurrent PCIe Memory Write traffic. This path represents the direction in which host-side peripherals write data into DUT-visible FPGA memory. Architectural analysis of pre-silicon prototyping environments motivates such concurrent stress testing to expose arbitration bottlenecks and memory-access saturation [?]. Accordingly, we devise a Host-Injected Traffic Stress Test that generates multiple independent PCIe Memory Write streams into distinct FPGA BAR regions, stressing the FPGA internal interconnect, address-translation logic, and memory-write arbitration.

Methodology: We developed a custom multi-threaded micro-benchmark to generate independent synthetic DMA ingress streams from multiple host threads. Each thread performs continuous bulk transfers to a distinct FPGA BAR offset, creating concurrent PCIe Memory Write transactions

figures/FPGA.drawio.png

Fig. 7. 128K Sequential Read/Write Bandwidth and 4K Random Read/Write IOPS Comparison.

that model Memory-Write ingress traffic generated by up to 16 independent PCIe peripherals.

Result Analysis: As shown in Figures ?? and ??, per-stream latency increases under PCIe and AXI contention, while the system remains stable for up to 16 concurrent Memory Write streams. Aggregated Memory-Write ingress throughput scales approximately linearly from 1 to 8 streams and then converges toward the Memory-Write ingress-path saturation point of approximately **1200 MB/s** at 16 streams.

D. End-to-End System Evaluation

As illustrated in Figure ??, the SPDK-driven end-to-end evaluation shows that the ARM hard core achieves 1667.56/911.87 MB/s for sequential reads/writes and 266.6/249.5 kIOPS for random reads/writes. The 100 MHz RocketChip soft core achieves 332.02/327.75 MB/s and 13.4/13.3 kIOPS, respectively. These results demonstrate end-to-end operation across both hard-core and soft-core DUTs; the lower RocketChip result is consistent with its 100 MHz execution environment.

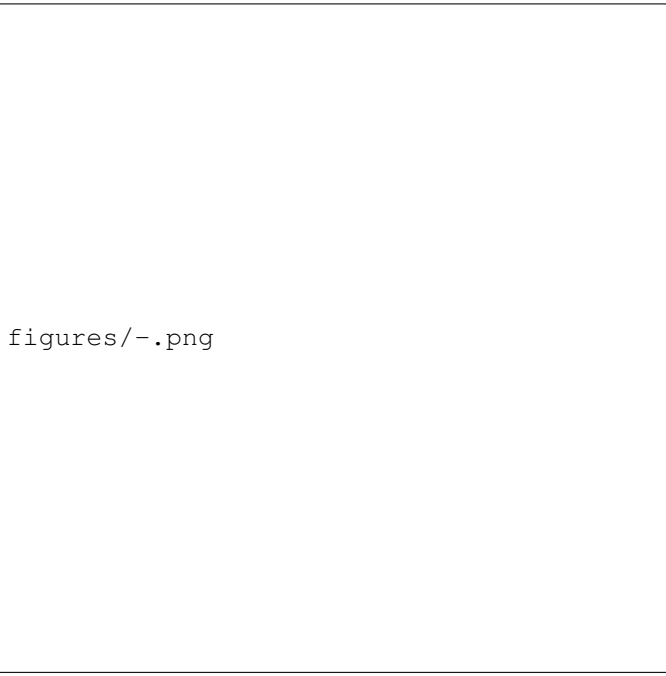
E. Case Study: Command-Level Debugging of NVMe Transactions

To demonstrate Proxy Forwarding Mode's practical utility, we use it to trace and debug cross-domain NVMe command submissions. Instead of observing opaque hardware faults when a device rejects an invalid DMA mapping, developers can inspect the exact progression of address translation and control-register propagation.



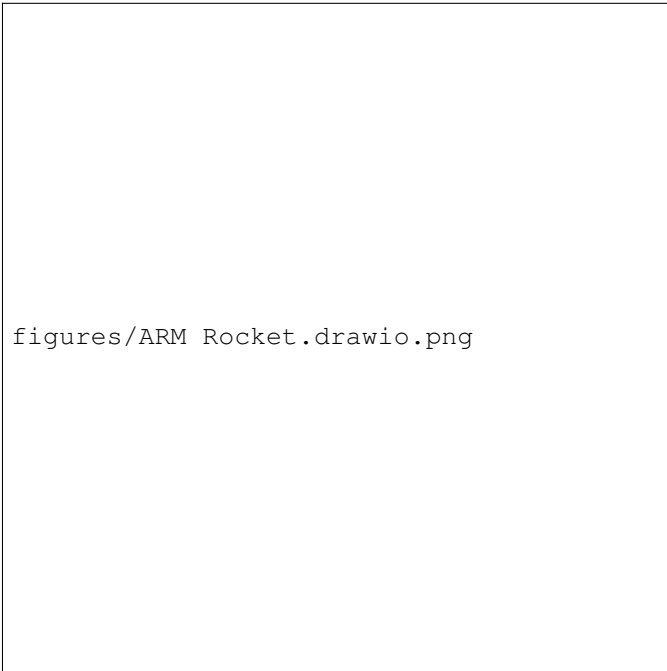
figures/v3/.png

Fig. 8. Impact of Thread Concurrency on Average Data Transfer Time. While latency increases with contention, the system remains stable without timeouts.



figures/- .png

Fig. 9. Aggregated Throughput Scalability for Synthetic Memory Write Ingress. Throughput scales approximately linearly from 1 to 8 synthetic ingress streams and approaches the write-ingress physical-path limit of about 1200 MB/s at 16 streams.



figures/ARM Rocket.drawio.png

Fig. 10. End-to-End 128K Sequential Bandwidth and 4K Random IOPS.

Figure ?? shows the real-time proxy trace during an NVMe command submission (e.g., Identify). The framework provides deep visibility into four progressive stages:

- 1) **Command Interception ([RAW]):** The proxy intercepts the raw command initialized by the DUT and extracts its original arguments (`prp1=0x8336a000`).
- 2) **Semantic Translation ([PATCHED]):** The FPGA routing substrate overwrites the device-level payload, translating the DUT-local PRP buffer address to the host-visible DUT memory address domain (`prp1=0xd2c8336a000`).
- 3) **Lifecycle Tracking ([TRACK]):** The system tracks the command ID (`cid=4108`) to ensure proper data visibility and completion writeback back to the DUT.
- 4) **Doorbell Replay ([BAR] [WR]):** The proxy forwards the Submission Queue doorbell write (`SQ0_DOORBELL`) to the physical peripheral, actively triggering execution.

This explicit control-path exposure allows developers to verify command construction, payload integrity, and cross-domain DMA mappings before physical device execution, serving as a powerful software-accessible debugging tool for driver validation.

VI. CONCLUSION

This work presents a reconfigurable FPGA routing substrate that connects processor DUTs to host-resident PCIe peripherals without dedicated FMC or NTB-based sharing equipment. SCOPE combines cross-domain address translation, two register-forwarding control modes (Direct Control Mode and Proxy Forwarding Mode), and DUT-visible interrupt

```
[CMD][RAW][ADDR] guest_pa=0x0000845ca000 bypass=0x0d2c845ca000
[CMD][RAW] opcode=0x06 cid=4108 prp1=0x00008336a000
[CMD][PATCHED] opcode=0x06 cid=4108 prp1=0x0d2c8336a000
[CMD][WRITEBACK][OK] guest_pa=0x0000845ca000
[CMD][ADMIN][TRACK] qid=0 slot=0 cid=4108
[BAR][WR] reg=SQ0_DOORBELL size=4 data=0x0000000100000001 OK
```

Fig. 11. Proxy Forwarding Mode Trace Log of an NVMe Transaction. The framework explicitly exposes raw command capture, PRP physical address translation, completion tracking, and hardware doorbell replay.

regeneration, while device-specific semantic adapters describe submission triggers, command metadata, DMA-address fields, completions, and event policy. Rather than transparently attaching remote devices to unmodified DUT drivers, Direct Control Mode uses DUT-side software and Proxy Forwarding Mode serializes control operations into observable command packets. For NVMe, SCOPE retains 99.1% of host-memory sequential-read bandwidth and 98.5% of random-read IOPS; the current write path retains 71.3% and 77.0%, leaving AXI write buffering as future optimization targets. The virtual INTx path delivers all 100 events without timeout with 956.81 ns mean host coordinator injection overhead. NIC experiments demonstrate BAR-control compatibility, and synthetic streams characterize shared FPGA write-ingress scalability.